task2

August 23, 2026

[16]:
```
import numpy as np

dataset = [
    {
        "filename": "story938.txt",
        "text": "Anger explodes in streets of Athens. The riot, widely␣
 ↪described as one of the worst in Athens since Greece began dealing with its␣
 ↪crippling debt crisis in 2010, broke out after the Greek Parliament approved␣
 ↪a new package of austerity measures in return for a new eurozone bailout of␣
 ↪the debt-stricken country...",
        "words": [
            "explodes", "street", "riot", "widely", "describe", "one", "bad",␣
 ↪"since",
            "begin", "deal", "cripple", "debt", "crisis", "break", "approve",␣
 ↪"new",
            "package", "austerity", "measure", "return", "new", "eurozone",␣
 ↪"bailout",
            "country", "among", "crowd", "estimate", "police", "hurl", "rock",␣
 ↪"toward",
            "police", "respond", "tear", "gas", "least", "people", "arrested",␣
 ↪"police",
            "say", "say", "investigator", "still", "tally", "property",␣
 ↪"damage", "say",
            "building", "damage", "fire", "numerous", "others", "loot",␣
 ↪"damaged",
            "building", "bank", "cafes", "movie", "theater", "news", "agency",␣
 ↪"report",
            "violent", "protest", "reflect", "increase", "angst", "crush",␣
 ↪"economic",
            "problem", "say", "protest", "feel", "end", "financial", "suffer",␣
 ↪"people",
            "every", "day", "get", "poor", "become", "homeless", "especially",␣
 ↪"say",
            "scatter", "protester", "tear", "gas", "package", "include",␣
 ↪"deep", "cut",
            "government", "spending", "wage", "pension", "help", "pave", "way",
```

```
          "eurozone", "finance", "minister", "sign", "new", "billion",␣
↪"billion",
          "bailout", "deal", "pass", "vote", "need", "fund", "order", "meet",
          "billion", "debt", "repayment", "due", "next", "month", "vote",␣
↪"call",
          "calm", "vandalism", "violence", "place", "democracy", "tolerate",␣
↪"told",
          "lawmaker", "urge", "approve", "deal", "critical", "time", "luxury",
          "conflict", "say", "comment", "make", "speech", "even", "warn",␣
↪"lawmaker",
          "government", "would", "able", "pay", "salary", "cover", "service",
          "import", "basic", "good", "plan", "approve", "spoke",␣
↪"possibility",
          "catastrophic", "bankruptcy", "hard", "program", "tough", "program",
          "also", "envisage", "painful", "sacrifice", "say", "add", "plan",␣
↪"would",
          "put", "back", "stable", "ground", "criticize", "program",␣
↪"demonstrate",
          "day", "eurozone", "figure", "stack", "isolated", "break",␣
↪"protester",
          "rally", "front", "building", "mood", "calmer", "day", "earlier",␣
↪"protest",
          "disperse", "youth", "smash", "pavement", "begin", "throw",␣
↪"stone", "piece",
          "marble", "well", "cocktails", "police", "respond", "stun",␣
↪"grenade",
          "protest", "widely", "describe", "bad", "since", "debt", "crisis",␣
↪"begin",
          "year", "three", "people", "die", "fire", "bomb", "hit", "bank",␣
↪"central",
          "protest", "austerity", "measure", "though", "sweeping", "reform",␣
↪"package",
          "agree", "troika", "make", "approved", "lawmaker", "must", "still",␣
↪"prime",
          "minister", "head", "bring", "together", "finance", "minister",␣
↪"say",
          "assurance", "also", "need", "bailout", "could", "pay", "political",
          "leader", "must", "pledge", "continue", "implement", "measure",␣
↪"upcoming",
          "election", "say", "must", "also", "find", "million", "structural",
          "expenditure", "cut", "add", "bailout", "deal", "would", "result",
          "significant", "loss", "bondholder", "intend", "help", "reduce",␣
↪"debt",
          "gross", "domestic", "product", "currently", "owe", "billion",␣
↪"come",
```

```
            "close", "default", "nation", "struggle", "follow", "austerity",␣
 ↪"measure",
            "economic", "reform", "condition", "bailout", "package", "time",␣
 ↪"economy",
            "recession", "year", "many", "analyst", "warn", "additional",␣
 ↪"austerity",
            "could", "make", "situation", "bad", "stock", "higher", "morning",␣
 ↪"follow",
            "passage", "austerity", "package", "market", "anxious", "potential",
            "impact", "default", "world", "economy"
        ]
    }
]



def makeDocumentTermMatrix(data):

    vocabulary = {}
    for doc in data:
        for word in doc["words"]:
            if word not in vocabulary:
                vocabulary[word] = len(vocabulary)

    W = len(vocabulary)
    D = len(data)
    matrix = np.zeros((W, D))

    for j, doc in enumerate(data):
        for word in doc["words"]:
            i = vocabulary[word]
            matrix[i, j] += 1

    index_to_word = {i: w for w, i in vocabulary.items()}
    return matrix, vocabulary, index_to_word

def log_entropy_normalization(matrix):
    W, D = matrix.shape

    local_weight = np.log2(1 + matrix)

    if D <= 1:
        print("Only 1 document found. Cannot normalize")
        return local_weight

    rows_sums = matrix.sum(axis=1, keepdims=True)
```

```
    p_ij = np.divide(matrix, rows_sums, out=np.zeros_like(matrix),␣
 ↪where=rows_sums!=0)

    entropy_elements = np.zeros_like(p_ij)
    mask = p_ij > 0
    entropy_elements[mask] = p_ij[mask] * np.log2(p_ij[mask])

    row_entropy = entropy_elements.sum(axis=1, keepdims=True)
    global_weight = 1 + (row_entropy / np.log2(D))

    return local_weight * global_weight

raw_matrix, vocabulary, index_to_word = makeDocumentTermMatrix(dataset)
scaled_matrix = log_entropy_normalization(raw_matrix)
```

```
Only 1 document found. Cannot normalize
```

[17]:
```
U, sigma, Vt = np.linalg.svd(scaled_matrix, full_matrices=False)

k = 2

U_k = U[:, :k]
Sigma_k = np.diag(sigma[:k])
Vt_k = Vt[:k, :]

projected_words = np.dot(scaled_matrix, Vt_k.T)
```

[18]:
```
def find_closest_words(target_word, n=3):
    if target_word not in vocabulary:
        return f"'{target_word}' not found in the vocabulary."

    target_idx = vocabulary[target_word]
    target_vector = projected_words[target_idx]

    target_norm = np.linalg.norm(target_vector)
    if target_norm == 0:
        return "Target word vector is zero."

    similarities = {}

    for i in range(len(vocabulary)):
        if i == target_idx:
            continue

        compare_vector = projected_words[i]
        compare_norm = np.linalg.norm(compare_vector)
```

```
        if compare_norm == 0:
            continue

        dot_product = np.dot(target_vector, compare_vector)
        cos_sim = dot_product / (target_norm * compare_norm)

        similarities[index_to_word[i]] = cos_sim

    sorted_sims = sorted(similarities.items(), key=lambda item: item[1],␣
↪reverse=True)
    return sorted_sims[:n]


test_word = "explodes"
print(f"Top 3 closest words to '{test_word}':")
results = find_closest_words(test_word, n=3)

for rank, (word, score) in enumerate(results, 1):
    print(f"{rank}. {word} (Similarity: {score:.4f})")
```

```
Top 3 closest words to 'explodes':
1. street (Similarity: 1.0000)
2. riot (Similarity: 1.0000)
3. widely (Similarity: 1.0000)
```